

Embedded Software Engineer Task

We'd like this task to take you around 3-4 hours: please don't feel you need to spend longer on it.

The goal is to have you work through a task using the micro:bit C++ APIs and CODAL so that we have something concrete to talk through in the interview. We want to be able to assess your technical abilities but without putting you on the spot or doing abstract whiteboard coding exercises.

If you are called to interview, we'd like you to bring to the interview a laptop on which you can share and build this code, so we can look at it together and discuss it. One of the interviewers will be connected via Zoom so we'd like to be able to share your screen.

We don't mind you using an AI assistant to help with the task, but we'd like to know what and how. Please be explicit when you submit the task what was generated by any AI assistant.

We also suggest that you ensure you understand the code you submit as we plan to ask you about it.

Part 1: Introducing new events

In this task, we'd like you to add new events, and build a music box program that consumes them.

The micro:bit uses an event-based model for lots of the things a user can do - for example the button presses and the "shake" event on the accelerometer that you can use in MakeCode.

The event-based system you see in [MakeCode](#) sits directly on top of the C++ one built in CODAL.

Looking over the micro:bit APIs, some of our sensors lack these events: for example there's no "on light" or "on dark" event in MakeCode, or in CODAL for micro:bit.

This should be done in such a way that they would, in the future, be easily added to the MakeCode and MicroPython APIs.

The micro:bit light sensing is deeply intertwined with the MicroBitDisplay because we use the LED's from the display in a reverse bias configuration to sense light level.

Build a “music box”

Please build a micro:bit program to allow you to make a “micro:bit music box” by putting a micro:bit and battery pack inside a box with a lid that opens and closes. It should behave as much like a real music box as possible, except that it should use the presence or absence of light to determine when to start and stop playing.

[Here's a quick video of someone using the kind of music box we're thinking about.](#)

Please pay attention to the details of how a music box works and sounds when creating your example. Unless you want to, don't worry about the wind-up mechanism!

Part 2: draw a flow diagram of the default audio pipeline

Having built something that plays music, please now describe the audio pipeline.

Can you draw a block diagram for the CODAL audio pipeline that is instantiated as `MicroBitAudio` in the `MicroBit` object of the repo here

<https://github.com/lancaster-university/codal-microbit-v2> - what's the meaning of push/pull and what is hooked up initially to the mixer?

Deliverables

Please share the following with us by the deadline we provide in the email.

- Forks of the respective CODAL GitHub repositories with this feature implemented, and the music box programme.
 - If you prefer not to use GitHub, any other platforms or version control systems are also acceptable, although less preferable. Alternatively we could accept a zip file with the source code repository. It's useful for us to see the commits you make.
- The diagram of the audio pipeline.

We're happy to receive other assets or things you'd like to share that help contextualise or explain your solution.

If you have any specific open source project, contribution, or personal work you'd like to share with us, please do include a link! We're interested in seeing real-world examples of your work and contributions.

Information & useful links

The entry point to create a micro:bit C++ programme is the micro:bit V2 samples repository:

<https://github.com/lancaster-university/microbit-v2-samples>

This repository `build.py` script will download the CODAL micro:bit V2 target to the `libraries` folder, and also its required libraries in the same folder, as indicated in this JSON file:

<https://github.com/lancaster-university/codal-microbit-v2/blob/master/target-locked.json#L54-L73>

If you google "CODAL documentation" you might find [this repository](#), or the [V1 DAL docs](#). The V1 docs do not cover the micro:bit V2 CODAL functionality, and the docs repository is a work-in-progress project to build a static page from the source code docs. Our main recommendation is to navigate through the source code and their docstrings directly.